

YOCTO PROJECT LAB

Two Simple Native C++ Recipes

Student Handout, Practical Tasks, and Review Questions



Learning focus

Compile a host-native C++ application using BitBake.

Create a custom task that runs the application during the build.

Keep the executable and generated output visible in `${WORKDIR}`.

Understand the difference between `${WORKDIR}`, `${D}`, and `${bindir}`.

Name: _____ Date: _____

Layer name: _____ Yocto release: _____

Lab Overview

In this lab, you will create two recipes. Both recipes use the native class, so the generated programs run on the development computer during the BitBake build.

Important: `${WORKDIR}` is a temporary recipe work area. The files remain visible after a normal build, but clean tasks can remove them. Copy required submissions before cleaning the recipe.

Required layer structure

```
meta-student/
  recipes-student/
    hello-native/
      hello-native_1.0.bb
      files/
        main.cpp
    student-score-native/
      student-score-native_1.0.bb
      files/
        score.cpp
```

Useful commands

```
# Build a recipe
bitbake hello-native

# Print the exact WORKDIR path
bitbake -e hello-native | grep '^WORKDIR='

# Re-run only the custom run task
bitbake -c run -f hello-native

# Remove the recipe work directory
bitbake -c clean hello-native
```

Why use `${CXX}`?: BitBake supplies the correct compiler and build environment through variables such as `${CXX}`, `${CXXFLAGS}`, and `${LDFLAGS}`. Do not hard-code g++ inside a recipe.

Expected workflow

1. Create the recipe directory and files inside your custom layer.
2. Add your layer to `bblayers.conf` if it is not already enabled.
3. Build the recipe with BitBake.
4. Use `bitbake -e` to locate `${WORKDIR}`.
5. Open the student-output directory and inspect the executable and text output.
6. Modify the source or recipe, rebuild, and compare the output.

Exercise 1 - Hello Native

Goal: compile a simple C++ program, run it during the build, and redirect its console output to a text file under \${WORKDIR}/student-output.

1. Create the C++ source file

recipes-student/hello-native/files/main.cpp

```
#include <iostream>

int main()
{
    std::cout << "Hello from a Yocto native recipe!" << std::endl;
    std::cout << "The application ran on the build computer." << std::endl;
    return 0;
}
```

2. Create the recipe

recipes-student/hello-native/hello-native_1.0.bb

```
SUMMARY = "Simple native C++ hello application"
LICENSE = "CLOSED"

SRC_URI = "file://main.cpp"
S = "${WORKDIR}"

inherit native

do_compile() {
    install -d ${WORKDIR}/student-output

    ${CXX} ${CXXFLAGS} ${LDFLAGS} \
        ${S}/main.cpp \
        -o ${WORKDIR}/student-output/hello-native
}

do_run() {
    ${WORKDIR}/student-output/hello-native \
        > ${WORKDIR}/student-output/hello-output.txt
}

addtask run after do_compile before do_install

do_install() {
    install -d ${D}${bindir}
    install -m 0755 \
        ${WORKDIR}/student-output/hello-native \
        ${D}${bindir}/hello-native
}
```

3. Build and inspect the result

```
bitbake hello-native
bitbake -e hello-native | grep '^WORKDIR='

# Then inspect:
# <WORKDIR>/student-output/hello-native
# <WORKDIR>/student-output/hello-output.txt
```

Expected hello-output.txt

```
Hello from a Yocto native recipe!
The application ran on the build computer.
```

Exercise 1 - Student Questions

1. What does inherit native change about the application and where it runs?

2. Why should a recipe use `${CXX}` instead of writing `g++` directly?

3. Which task creates the hello-native executable?

4. Which task creates hello-output.txt?

5. Write the exact `${WORKDIR}` path reported by your build.

6. Explain the meaning of: `addtask run after do_compile before do_install`.

Practical tasks

[] Change the C++ message to include your name and rebuild the recipe.

[] Rename hello-output.txt to my-first-output.txt in the recipe.

[] Add a third output line that states the name of your custom layer.

[] Force only the run task again using `bitbake -c run -f hello-native`.

[] Show the executable and output file to the instructor inside `${WORKDIR}/student-output`.

Record your result

7. What changed in the output after your modifications?

Exercise 2 - Student Score Report

Goal: compile a native C++ program that accepts three command-line scores, calculates the average, and writes the result to a file during the build.

1. Create the C++ source file

`recipes-student/student-score-native/files/score.cpp`

```
#include <cstdlib>
#include <iomanip>
#include <iostream>

int main(int argc, char* argv[])
{
    if (argc != 4) {
        std::cerr << "Usage: student-score <quiz> <lab> <exam>" << std::endl;
        return 1;
    }

    const int quiz = std::atoi(argv[1]);
    const int lab = std::atoi(argv[2]);
    const int exam = std::atoi(argv[3]);
    const double average = (quiz + lab + exam) / 3.0;

    std::cout << "Student Score Report" << std::endl;
    std::cout << "Quiz: " << quiz << std::endl;
    std::cout << "Lab: " << lab << std::endl;
    std::cout << "Exam: " << exam << std::endl;
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Average: " << average << std::endl;
    std::cout << "Result: " << (average >= 60.0 ? "PASS" : "FAIL") << std::endl;

    return 0;
}
```

2. Create the recipe

recipes-student/student-score-native/student-score-native_1.0.bb

```
SUMMARY = "Native C++ student score report"
LICENSE = "CLOSED"

SRC_URI = "file://score.cpp"
S = "${WORKDIR}"

inherit native

QUIZ_SCORE ?= "80"
LAB_SCORE ?= "90"
EXAM_SCORE ?= "70"

do_compile() {
    install -d ${WORKDIR}/student-output

    ${CXX} ${CXXFLAGS} ${LDFLAGS} \
        ${S}/score.cpp \
        -o ${WORKDIR}/student-output/student-score
}

do_run() {
    ${WORKDIR}/student-output/student-score \
        ${QUIZ_SCORE} ${LAB_SCORE} ${EXAM_SCORE} \
        > ${WORKDIR}/student-output/score-result.txt
}

addtask run after do_compile before do_install

do_install() {
    install -d ${D}${bindir}
    install -m 0755 \
        ${WORKDIR}/student-output/student-score \
        ${D}${bindir}/student-score
}
```

3. Build and inspect the result

```
bitbake student-score-native
bitbake -e student-score-native | grep '^WORKDIR='

# Then inspect:
# <WORKDIR>/student-output/student-score
# <WORKDIR>/student-output/score-result.txt
```

Expected score-result.txt

```
Student Score Report
Quiz: 80
Lab: 90
Exam: 70
Average: 80.00
Result: PASS
```

Exercise 2 - Student Questions

1. Why does the program expect argc to equal 4 even though only three scores are entered?

2. What does the > symbol do in the do_run task?

3. What average do you expect from scores 80, 90, and 70?

4. Where are QUIZ_SCORE, LAB_SCORE, and EXAM_SCORE defined?

5. What is the benefit of using = when defining the score variables?</p

6. What result should be generated for scores 40, 50, and 60? Explain.

Practical tasks

[] Change the three score variables and confirm that score-result.txt changes.

[] Change the pass mark from 60 to 65 in the C++ source.

[] Add a fourth command-line value called attendance.

[] Add a line that prints the highest score.

[] Create a second task that generates score-result-low.txt using low scores.

Record your modified output

Compare the Two Recipes

Topic	Hello Native	Student Score Native
Input	No command-line input	Three score arguments
Generated file	hello-output.txt	score-result.txt
Main learning point	Task order and redirection	Recipe variables and arguments
Executable location	\${WORKDIR}/student-output	\${WORKDIR}/student-output
Installed location	\${D}\${bindir}	\${D}\${bindir}

Final review questions

1. Describe the difference between \${WORKDIR}, \${D}, and \${bindir}.

2. Why is student-output created with install -d before compilation?

3. What would happen if addtask run were removed from the recipe?

4. Why is the executable copied to \${D}\${bindir} during do_install?

5. Which files are source inputs and which files are build outputs in each exercise?

6. What can remove the files stored in \${WORKDIR}?

Challenge Tasks and Submission

Choose at least two challenge tasks.

- ☐ Move the compilation command into a simple Makefile and update SRC_URI.
- ☐ Split the C++ application into main.cpp, report.cpp, and report.hpp.
- ☐ Add a recipe variable called STUDENT_NAME and pass it as a compiler definition.
- ☐ Generate a CSV output file instead of a plain text report.
- ☐ Add input validation so scores below 0 or above 100 return an error.
- ☐ Add a do_check task that verifies the expected text exists in the output file.

Submission checklist

- ☐ Both recipes build successfully without errors.
- ☐ Both executables exist under their recipe \${WORKDIR}/student-output directory.
- ☐ Both text output files exist and contain the expected result.
- ☐ All student questions are answered.
- ☐ At least two practical modifications are completed.
- ☐ The exact \${WORKDIR} paths are recorded before running any clean task.

Instructor verification

Exercise 1 completed: ☐ Yes ☐ Needs revision Initials: _____

Exercise 2 completed: ☐ Yes ☐ Needs revision Initials: _____

Challenge completed: ☐ Yes ☐ Needs revision Initials: _____

Reminder: The executable in this lab is native to the build computer. It is not an application for the target image. A target recipe would omit inherit native and would normally install the program into the target root filesystem.

End of Lab